

Alocação de Computação de Inferência na Síntese de Programas com Modelos de Linguagem

Anonymous submission

Resumo

Um orçamento fixo de chamadas a um modelo de linguagem pode ser gasto de dois modos: gerando N candidatos independentes ou revisando um único candidato com feedback. A literatura relata tanto ganhos quanto falhas para a segunda via, e separá-las é difícil porque, sem informação externa, o revisor não sabe que errou. Damos a essa via uma vantagem que ela não teria em produção — um Crítico que enxerga o gabarito do par de teste, mas nunca seleciona a resposta. Em 270 tarefas do ARC-AGI-1 sob orçamento igualado, **nenhuma** das 5 comparações pré-registradas é significativa. Um segundo achado independe desse resultado: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas, e descontá-la inverte a ordenação entre as condições. Por subamostragem, em 30 tarefas essa ordenação se reproduz em 8% das vezes.

1 Introdução

Ganhos recentes em raciocínio vêm de escalar a computação no momento da inferência, e não os parâmetros do modelo (Snell et al. 2024; Brown et al. 2024). Isso levanta uma questão de custo direto: dado um orçamento fixo de chamadas, como alocá-lo?

Há duas respostas. **Diversificar** é gerar N programas independentes e selecionar o melhor por um verificador barato — a receita de *pass@k* em geração de código (Chen et al. 2021) e do AlphaCode (Li et al. 2022). **Iterar** é gastar o mesmo orçamento revisando um único candidato à luz de feedback, como em *self-debugging* (Chen et al. 2024), SELF-REFINE (Madaan et al. 2023) e REFLEXION (Shinn et al. 2023).

A evidência sobre a segunda via é contraditória: os trabalhos originais relatam ganhos expressivos, mas réplicas controladas encontram que grande parte deles desaparece quando o orçamento é igualado ou quando o modelo precisa detectar sozinho que errou (Huang et al. 2024; Stechly, Marquez, and Kambhampati 2023; Olausson et al. 2024). Uma explicação é que autocrítica não é crítica: sem informação externa sobre o próprio erro, o modelo reafirma a resposta que já produziu. O limitante seria o canal de feedback, não a estratégia iterativa.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Nosso Crítico enxerga o gabarito do par de teste, que o Gerador nunca vê — uma vantagem indisponível a qualquer sistema autônomo. O desenho é, assim, um **limite superior otimista** da via iterativa: se nem ele vencer, a vantagem atribuída ao feedback iterativo fica sob suspeita. Avaliamos no ARC-AGI-1 (Chollet 2019), em que cada tarefa traz pares entrada → saída de demonstração e exige inferir a transformação; a resposta é verificável exatamente e os pares de demonstração dão um verificador barato que ambas as estratégias usam sem tocar no gabarito.

2 Trabalho Relacionado

A via iterativa realimenta o modelo com o resultado da execução (Chen et al. 2024) ou com crítica gerada por ele mesmo (Madaan et al. 2023; Shinn et al. 2023). Huang et al. (2024) argumentam que modelos não corrigem raciocínio sem sinal externo; Stechly, Marquez, and Kambhampati (2023) mostram que a melhora aparente provém do verificador embutido no protocolo; Olausson et al. (2024) constatam que o auto-reparo em código só compensa quando o crítico é mais capaz que o gerador. Guiar síntese por contraexemplos antecede os modelos de linguagem (Solar-Lezama et al. 2006); nossa condição `critic_cegis` o transpõe para linguagem natural.

3 Metodologia

3.1 Formulação e Condições

Uma tarefa t é um par $(\mathcal{D}_t, (x_t^*, y_t^*))$, em que $\mathcal{D}_t$ são os pares de demonstração e (x_t^*, y_t^*) o par de teste. Um solucionador recebe $\mathcal{D}_t$ e x_t^* e produz um programa π em Python; a tarefa é resolvida se e somente se $\pi(x_t^*) = y_t^*$, com igualdade exata da grade. O recurso controlado é o **orçamento de chamadas** B por tarefa, somando todos os papéis:

$$\Delta = \mathbb{E}_t[\mathbf{1}[\mathcal{C} \text{ resolve } t]] - \mathbb{E}_t[\mathbf{1}[\mathcal{S} \text{ resolve } t]] > 0, \quad (1)$$

com $\mathcal{S}$ diversificando e $\mathcal{C}$ iterando. Manter B idêntico é o controle experimental central.

No laço da Figura 1, `sampling` aloca o orçamento em largura: cada uma das B chamadas é uma conversa nova com o mesmo *prompt*, sem histórico nem crítica, e vence o candidato que reproduz mais pares de demonstração. `critic` aloca em profundidade: o Gerador propõe regra e programa,

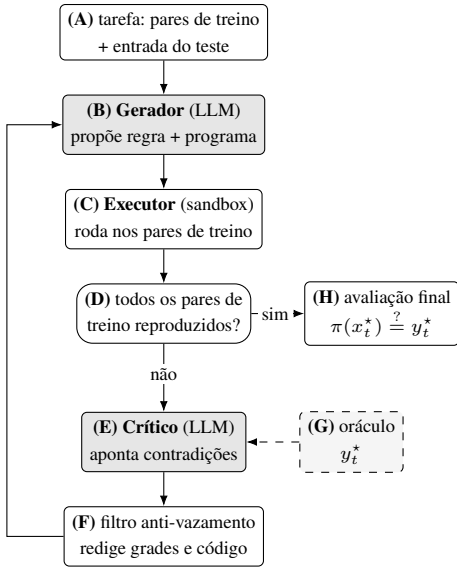


Figura 1: Arquitetura do solucionador. O Gerador (B) nunca vê y_t^* ; o Crítico (E) o recebe do oráculo (G), pela linha tracejada, apenas nas condições que o concedem, e sua saída passa pelo filtro (F) antes de voltar ao Gerador. A restrição central é que y_t^* não decide nada: o critério de parada (D) olha apenas para os pares de demonstração, e a avaliação final (H) roda uma única vez, ao término do laço, só para medir.

o Crítico aponta contradições em prosa livre, e o Gerador revisa com o histórico completo. Cada ciclo custa duas chamadas, de modo que com $B = 7$ resultam 3,23 programas contra os 5,43 da amostragem. *Best-of-N* é a linha de base porque nada obriga quem dispõe de B chamadas a gastá-las revisando um único programa em série.

Essa comparação é pragmática, não causal: as condições diferem em quatro eixos simultâneos — existência de feedback, acesso ao gabarito, orçamento em série ou paralelo, e temperatura (0,8 em *sampling*, 0,2 nas demais; sem essa assimetria as N amostras colapsariam em programas quase idênticos). Para separar os dois eixos de maior interesse, duas condições variam um único eixo em relação a *critic* (Figura 2): *critic_no_oracle* recebe o código, a regra e o relatório de execução, mas nunca y_t^* ; *critic_cegis* recebe o mesmo que *critic*, inclusive y_t^* , mas responde com um contraexemplo e uma classe de correção em vocabulário fechado em vez de prosa livre.

3.2 Salvaguardas e Análise

O gabarito **não seleciona**: o critério de parada é idêntico nas quatro condições — o candidato é aceito quando reproduz todos os pares de demonstração — e o par de teste roda uma única vez, ao final; se o laço parasse ao acertar o teste, a acurácia viraria um limite superior de oráculo. O gabarito **não vaza**: um filtro redige, na saída do Crítico, linhas compostas apenas de inteiros e qualquer código, e cada redação é contabilizada, tornando o vazamento mensurável. O Crítico **não resolve**: é reinstantiado a cada ro-

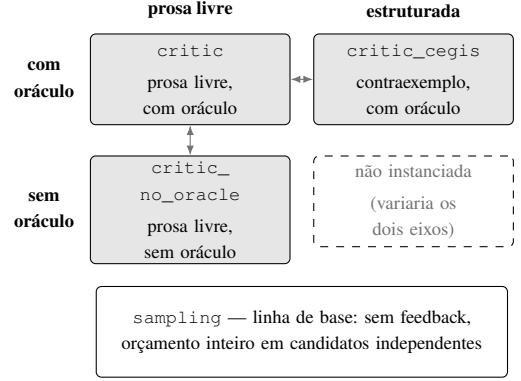


Figura 2: As quatro condições nos dois eixos da decomposição. As setas marcam as comparações que variam um único eixo: na horizontal, a forma do feedback com o acesso à informação constante; na vertical, o acesso com a forma constante.

Condição	Res.	Acur.	Treino	Prog.	Tok.
<i>critic</i>	93/270	34,4%	35,6%	3,23	48,1 k
<i>sampling</i>	88/270	32,6%	35,6%	5,43	28,0 k
<i>cegis</i>	83/270	30,7%	34,8%	3,34	48,8 k
<i>no_oracle</i>	80/270	29,6%	33,3%	3,27	45,8 k

Tabela 1: Resultados agregados, ordenados por acurácia. “Treino” é a fração de tarefas em que o candidato final reproduz todos os pares de demonstração — o alvo que as estratégias de fato otimizam; “Prog.” é o número médio de programas por tarefa e “Tok.”, o consumo médio de tokens.

dada, sem histórico, e não pode construir uma solução ao longo do diálogo. Gerador e Crítico usam o mesmo modelo, *gemini-3.5-flash-lite*, para que uma eventual vantagem não seja atribuível a um segundo modelo mais capaz.

Todas as condições rodam sobre as mesmas 270 tarefas sorteadas de *evaluation* com semente 20260814 e $B = 7$, declaradas antes da execução. Em desenho pareado, o que importa não é a acurácia agregada, mas quantas tarefas cada condição resolveu que a outra não: aplicamos o teste de McNemar exato (McNemar 1947), que descarta as concordâncias e avalia se as discordâncias se dividem por acaso. Os intervalos são de Wilson (Wilson 1927) sobre a proporção de discordâncias a favor da intervenção, convertidos para pontos percentuais de acurácia; reportá-los distingue a ausência de efeito detectado da ausência de capacidade de medição. A família de 5 comparações (cada crítico contra *sampling* e *critic* contra os outros dois) foi pré-registrada, o que exige a correção de Bonferroni (Dunn 1961): $\alpha = 0,01$ por teste.

4 Resultados

4.1 Comparação em Escala

Nas mesmas 270 tarefas (Tabela 1), a hipótese da Equação 1 não se confirma: **nenhuma** das 5 comparações pré-registradas é significativa, nem sob Bonferroni nem a $\alpha =$

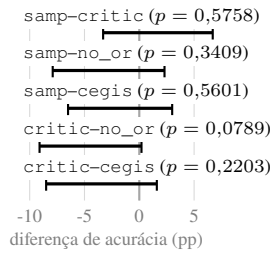


Figura 3: Intervalos de Wilson de 95% das 5 comparações pré-registradas, em pontos percentuais; a linha vertical marca a ausência de efeito e todos os intervalos a cruzam. As discordâncias, de cima para baixo, são 51 (23×28), 54 (31×23), 47 (26×21), 47 (30×17) e 54 (32×22). O que a figura evidencia é a largura dos intervalos: mesmo em 270 tarefas, a medição não separa as hipóteses.

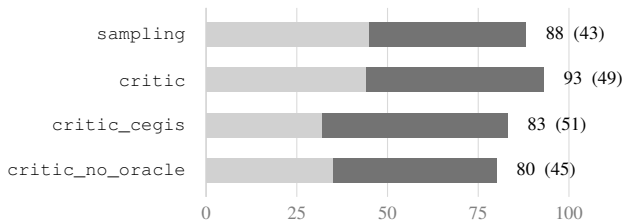


Figura 4: Decomposição das vitórias, em tarefas resolvidas de 270. O segmento claro são as decididas na primeira geração, comum às quatro condições; o escuro, as atribuíveis ao ciclo. Os rótulos dão o total e, entre parênteses, a parcela do ciclo. A ordenação pelo total (93, 88, 83, 80) *inverte* quando se olha apenas o ciclo (51, 49, 45, 43).

0,05 isolado (Figura 3). A comparação mais próxima é *critic* vs. *critic_no_oracle*, com $p = 0,0789$ e intervalo de $-9,1$ a $+0,2$ pp; a diferença entre *critic* e *sampling* é de $+1,9$ pp.

O mecanismo funciona no nível para que foi projetado: o Crítico produz 3,23 programas por tarefa contra 5,43 da amostragem — 40% a menos — com a *mesma* consistência nos pares de demonstração (35,6% nas duas condições). Essa economia de tentativas não vira acurácia no teste. Entre as quatro, *critic_no_oracle* tem a menor acurácia bruta e é a única que descreve um sistema executável sem gabarito.

4.2 A Origem das Vitórias

A primeira chamada é idêntica nas quatro condições — mesmo *prompt*, histórico vazio, nenhum crítico envolvido —, diferindo apenas na temperatura; vitórias decididas ali não distinguem estratégia alguma.

A Figura 4 mostra que cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas. Restringindo cada par às tarefas que a primeira geração não resolveu em nenhuma das duas condições, a medição discrimina em cerca de um quinto da amostra e todos os 5 p -valores ficam entre 0,2962 e 1,0000.

O poder efetivo de uma comparação entre arquiteturas de agentes é, portanto, bem menor do que n sugere. E descon-

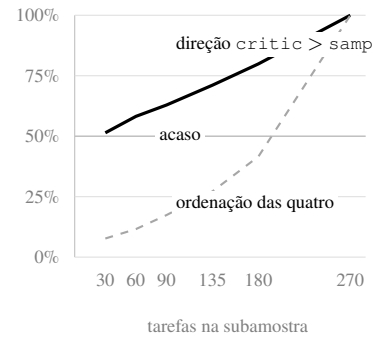


Figura 5: Reprodutibilidade da ordenação sob subamostragem das próprias 270 tarefas (20.000 repetições por tamanho, sem reposição, preservando o pareamento). A linha cheia é a fração de subamostras em que *critic* supera *sampling*; a tracejada, a fração em que a ordenação completa das quatro coincide com a observada em 270. Ambas convergem a 100% em 270 por construção.

tada a primeira geração, as três condições de crítico ficam *acima* de *sampling* em vitórias do ciclo (51, 49 e 45 contra 43) — direção oposta à do número bruto, com diferenças pequenas e nenhuma significativa.

O efeito replica numa medição independente sobre as mesmas tarefas, restrita a *sampling* e *critic*: 44 das 88 vitórias de *sampling* e 42 das 86 de *critic* vêm da primeira geração, deixando o placar do ciclo empatado em 44×44 ; nas 206 tarefas restantes a comparação também não distingue as condições ($p = 0,8877$, intervalo de $-5,6$ a $+4,3$ pp). Sob orçamento igualado, portanto, a revisão guiada por oráculo **não** superou a amostragem independente.

O confundimento é eliminável por construção: basta que a primeira condição a alcançar uma tarefa grave a primeira geração e as demais a repitam, com a repetição ainda debitada do orçamento. Nenhuma das medições aqui reportadas o faz.

4.3 Reprodutibilidade da Ordenação

As comparações acima não são significativas, mas a ordenação que produzem é frequentemente o que sobra de um estudo desses. Subamostramos então o conjunto fechado de 270 tarefas sem reposição, preservando o pareamento, em 20.000 repetições por tamanho (Figura 5).

Com 30 tarefas, a direção observada em 270 se reproduz em 51% das subamostras — indistinguível de um sorteio — e a banda de 5 a 95% da diferença de acurácia vai de $-10,0$ a $+13,3$ pp; com 60, ainda 58%. A ordenação completa das quatro condições é mais frágil: 8% e 12%. Estudos desta classe conduzidos em algumas dezenas de tarefas não estão medindo direção. Como a subamostragem capta apenas variância de *amostra*, esses valores são um piso otimista.

Extrapolar antes do fechamento tem efeito parecido: sobre as primeiras 91 tarefas, projetar linearmente as proporções da comparação mais próxima dava $p = 0,0027$, que teria atravessado a correção de Bonferroni, contra $p = 0,0789$ no conjunto fechado (Button et al. 2013).

5 Discussão e Limitações

Como o desenho favorece a via iterativa por construção, o resultado nulo indica que a informação privilegiada, isoladamente, não é o fator limitante. Não sustenta, porém, que a assimetria de informação seja irrelevante: os intervalos admitem efeitos que a medição não detectaria.

O confundimento da primeira geração não é específico deste arranjo — qualquer comparação entre esquemas que partam do mesmo *prompt* o herda. Contabilizamos orçamento em chamadas; em tokens, as condições de crítico consomem de 63% a 74% mais por tarefa, e o sinal poderia inverter.

Limitações. (i) Metade do resultado vem de uma chamada comum às condições, com correção conhecida e não aplicada. (ii) `critico` e `critico_cegis` consultam o gabarito durante o laço e medem a validação por oposição como limite superior, não um método implantável. (iii) A linha de base foi coletada em data anterior às condições de crítico, e o modelo é hospedado: as comparações com `sampling` carregam um confundimento temporal que as comparações entre críticos não têm. (iv) Uma execução por tarefa — o pareamento mitiga o ruído do modelo, não o elimina. (v) As conclusões referem-se ao regime de capacidade de `gemin-3.5-flash-lite`.

6 Conclusão

Em 270 tarefas do ARC-AGI-1 sob orçamento fixo, quatro condições que variam isoladamente o acesso ao oráculo e a forma do feedback não se separam: **nenhuma** das 5 comparações pré-registradas é significativa — embora o crítico atinja a mesma consistência com os pares de demonstração usando 40% menos programas. O achado mais transferível independe disso: cerca de metade das vitórias de qualquer condição é decidida na primeira geração, que é idêntica entre elas, e descontá-la inverte a ordenação — que se repete em apenas 12% das subamostras de 60 tarefas. O passo seguinte não é aumentar n , e sim eliminar esse confundimento e medir o orçamento em tokens.

Referências

Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q. V.; Ré, C.; and Mirhoseini, A. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.

Dunn, O. J. 1961. Multiple Comparisons Among Means. *Journal of the American Statistical Association*, 56(293): 52–64.

Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 46534–46594.

McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.

Olausson, T. X.; Inala, J. P.; Tan, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? In *The Twelfth International Conference on Learning Representations (ICLR)*.

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 8634–8652.

Snell, C.; Lee, J.; Xu, K.; and Kumar, A. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.

Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 404–415.

Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn't Know It's Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.